

Machine Learning

Clustering

Fabio Vandin

December 17th, 2025

Number of Iterations of Lloyd's Algorithm

- the number of iterations can be exponential in the input size: a trivial upper bound is $\approx k^m$ as before
- more sophisticated studies: upper bound $O(m^{kd})$ ($\mathbf{x} \in \mathbb{R}^d$)
- recent studies: lower bound $2^{\Omega(\sqrt{m})}$ in the worst-case
- in practice: much less than m iterations are required

Note: the convergence and the quality of the clustering depends on the initialization of the centers!

Effective Centers Initialization

Is there a way to choose the initial centers that is efficient but also provably leads to good clusters?

`k-means++`: simple but effective center initialization strategy proposed by D. Arthur and S. Vassilvitskii (article: D. Arthur and S. Vassilvitskii. *k-means++: the advantages of careful seeding*. Proc. of ACM-SIAM SODA 2007.)

Algorithm k-means++

$\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, with $\mathbf{x}_i \in \mathbb{R}^d$ for $1 \leq i \leq m$; $k \in \mathbb{N}^+$

Given a point $\mathbf{x} \in \mathcal{X}$ and a set F , let $d(\mathbf{x}, F) = \min_{\mathbf{f} \in F} d(\mathbf{x}, \mathbf{f})$

The algorithm to compute the initial set F of centers is the following:

$\mu_1 \leftarrow$ random point from $\mathcal{X}$ chosen uniformly at random;

$F \leftarrow \{\mu_1\}$;

for $i \leftarrow 2$ **to** k **do**

$\mu_i \leftarrow$ random point from $\mathcal{X} \setminus F$, choosing point $\mathbf{x}$ with
 probability $\frac{(d(\mathbf{x}, F))^2}{\sum_{\mathbf{x}' \in \mathcal{X} \setminus F} (d(\mathbf{x}', F))^2}$;
 $F \leftarrow F \cup \{\mu_i\}$;

return F ;

The following result is proved in the original paper by D. Arthur and S. Vassilvitskii.

Theorem

Let $\Phi_{k\text{-means}}^*(\mathcal{X}, k)$ be the cost of the optimal (i.e., minimum) k -means clustering of $\mathcal{X}$, and let $\Phi_{k\text{-means}}(\mathcal{X}, F_{k\text{-means}++})$ be the cost of the clustering $\mathcal{X}$ obtained by:

- using the points in $F_{k\text{-means}++}$ returned by $k\text{-means}++$ as centers;
- assigning each point of $\mathcal{X}$ to its closest center.

(Note that $\Phi(\mathcal{X}, F_{k\text{-means}++})$ is a random variable.) Then

$$\mathbb{E}[\Phi_{k\text{-means}}(\mathcal{X}, F_{k\text{-means}++})] \leq 8(\ln k + 2)\Phi_{k\text{-means}}^*(\mathcal{X}, k).$$

Notes:

- the expectation $\mathbb{E}[\Phi_{k\text{-means}}(\mathcal{X}, F_{k\text{-means++}})]$ is over all possible sets $F_{k\text{-means++}}$ returned by k-means++ (with input $\mathcal{X}$), which depends on the random choices in k-means++.
- k-means++ already provides a good solution for k -means, but it makes sense to use it to initialize centers in Lloyd's algorithm (the solution can only improve in the next iterations, if the first convergence criteria is used)

Choice of number k of clusters

Choosing the number k of clusters (e.g., for k -means) is not easy.

Common approach:

- 1 run clustering algorithm for various values of k , obtaining a clustering $C^{(k)} = \{C_1^{(k)}, C_2^{(k)}, \dots, C_k^{(k)}\}$ for each value of k considered;
- 2 use a score S to evaluate each clustering $C^{(k)}$, getting scores $S(C^{(k)})$ for each value of k
- 3 pick the value of k (and clustering) of maximum score:
$$C = \arg \max_{C^{(k)}} \{S(C^{(k)})\}$$

A very common score based on distances alone: *silhouette*

Silhouette

Given a clustering $C = (C_1, C_2, \dots, C_k)$ of $\mathcal{X}$ and a point $\mathbf{x} \in \mathcal{X}$, let $C(\mathbf{x})$ be the cluster to which $\mathbf{x}$ is assigned to. Assume $|C_i| \geq 2 \forall 1 \leq i \leq k$. Define:

$$A(\mathbf{x}) = \frac{\sum_{\mathbf{x}' \neq \mathbf{x}, \mathbf{x}' \in C(\mathbf{x})} d(\mathbf{x}, \mathbf{x}')}{|C(\mathbf{x})| - 1}$$

Given a cluster $C_i \neq C(\mathbf{x})$, let

$$d(\mathbf{x}, C_i) = \frac{\sum_{\mathbf{x}' \in C_i} d(\mathbf{x}, \mathbf{x}')}{|C_i|}$$

and $B(\mathbf{x}) = \min_{C_i \neq C(\mathbf{x})} d(\mathbf{x}, C_i)$.

Then the *silhouette* $s(\mathbf{x})$ of $\mathbf{x}$ is

$$s(\mathbf{x}) = \frac{B(\mathbf{x}) - A(\mathbf{x})}{\max\{A(\mathbf{x}), B(\mathbf{x})\}}$$

Intuition: $s(\mathbf{x})$ measures if $\mathbf{x}$ is closer to points in its “nearest cluster” than to the cluster it is assigned to.

Question: what is the range for $s(\mathbf{x})$?

The silhouette of clustering $C = (C_1, C_2, \dots, C_k)$ is

$$S(C) = \frac{\sum_{\mathbf{x} \in \mathcal{X}} s(\mathbf{x})}{|\mathcal{X}|}$$

The higher $S(C)$, the better the clustering quality.

Linkage-Based Clustering

General class of algorithms that follow the general scheme below.

Algorithm

- ① start from the trivial clustering: each data point is a (single-point) cluster
- ② **until “termination condition”**: repeatedly merge the “closest” clusters of the previous clustering

We need to specify two “parameters”:

- how to define distance between clusters
- termination condition

Linkage-Based Clustering (continue)

Different distances $D(A, B)$ between two clusters A and B can be used, resulting into different linkage methods:

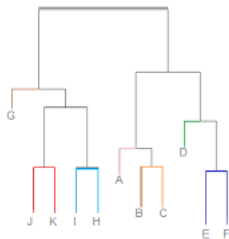
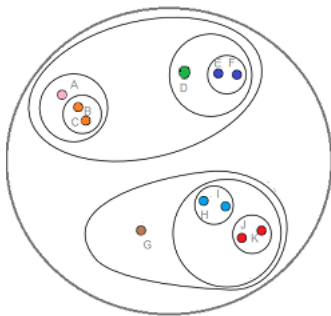
- **single linkage:** $D(A, B) = \min\{d(\mathbf{x}, \mathbf{x}') : \mathbf{x} \in A, \mathbf{x}' \in B\}$
- **average linkage:** $D(A, B) = \frac{1}{|A||B|} \sum_{\mathbf{x} \in A, \mathbf{x}' \in B} d(\mathbf{x}, \mathbf{x}')$
- **max linkage:** $D(A, B) = \max\{d(\mathbf{x}, \mathbf{x}') : \mathbf{x} \in A, \mathbf{x}' \in B\}$

Common termination condition:

- data points are partitioned into k clusters
- minimum distance between pairs of clusters is $> r$, where r is a parameter provided in input
- all points are in a cluster $\Rightarrow$ output is a dendrogram

Dendrogram: Example

Dendrogram: tree, with input points $\mathbf{x} \in \mathcal{X}$ as leaves, that shows the arrangement/relation between clusters.



Single Linkage Clustering: Algorithm

Input: data points $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$; termination condition;

Output: clustering of $\mathcal{X}$, with $C(\mathbf{x})$ being the cluster of $\mathbf{x}$;

for $i = 1, \dots, m$ **do** $C(\mathbf{x}_i) \leftarrow i$;

$Q \leftarrow$ empty priority queue;

for $i = 1, \dots, m - 1$ **do**

for $j = i + 1, \dots, m$ **do**

$Q.\text{insert}(d(\mathbf{x}_i, \mathbf{x}_j), (\mathbf{x}_i, \mathbf{x}_j));$

while *termination condition* **do**

$(k, (\mathbf{x}_i, \mathbf{x}_j)) \leftarrow Q.\text{removeMin}();$

if $C(\mathbf{x}_i) \neq C(\mathbf{x}_j)$ **then**

 /* merge the clusters of $\mathbf{x}_i$ and $\mathbf{x}_j$ */
 set $C(\mathbf{x}_\ell) = \min\{C(\mathbf{x}_i), C(\mathbf{x}_j)\}$ for all $\mathbf{x}_\ell$ in the clusters
 of $\mathbf{x}_i$ and $\mathbf{x}_j$;

return clustering C ;

Notes:

- the algorithm produces in output only the final clustering
- the algorithm can be easily modified to output the dendrogram, or the overall hierarchical clustering, etc.
- the algorithm is essentially a variant of Kruskal's algorithm for the Minimum Spanning Tree (MST) problem

Complexity?

- priority queue Q implemented as a heap
- compute all distances and initialize Q : time $\Theta(m^2)$
- **while** cycle
 - number of iterations? At most m^2
 - `removeMin()`: $O(\log m)$
 - merge clusters of x_i and x_j
 - naïve implementation: $O(m)$
 - using *union-find* data structures: $O(m^2 \log m)$, cumulative across iterations

$\Rightarrow$ complexity $O(m^2 \log m)$